Week-7

EXPERIMENT 1

SIMPLE WEB APPLICATION USING HTML

1. Aim

To create and execute a simple web application using HTML.

2. Description

HTML stands for HyperText Markup Language. It is used to create and design web pages. HTML contains different elements such as headings, paragraphs, buttons and forms.

An HTML web application can be executed directly in a web browser without requiring a servlet container.

3. Software Requirements

- Notepad or Eclipse IDE
- Google Chrome
- HTML

4. Step-by-Step Procedure

1. Open Notepad or Eclipse IDE.
2. Create a new file named index.html.
3. Write the HTML program.
4. Save the file with the .html extension.
5. Locate the saved file.
6. Open the file using Google Chrome.
7. Observe the webpage displayed in the browser.
8. Click the button to check the alert message.

5. Program

File Name: index.html

HTML

Wrap linesCopy codePreview

```
<!DOCTYPE html>
<html>
<head>
   <title>Simple Web Application</title>
</head>
<body>
   <h1>Welcome to My Web Application</h1>
   <h2>Student Information</h2>
   <p>Name: Rahul</p>
   <p>Course: Java Programming</p>

   <button onclick="alert('Welcome Student!')">
      Click Here
   </button>
</body>
</html>
```

6. Expected Output

The browser displays:

Welcome to My Web Application

Student Information

Name: Rahul

Course: Java Programming

[Click Here]

After clicking the button, the following alert appears:

Welcome Student!

7. Screenshots

Screenshot 1: HTML Source Code

Capture the index.html program in Notepad or Eclipse.

[Insert Screenshot Here]

Screenshot 2: HTML Browser Output

Capture the webpage displayed in Google Chrome.

[Insert Screenshot Here]

Screenshot 3: Button Alert

Capture the alert message after clicking the button.

[Insert Screenshot Here]

8. Result

Thus, a simple web application was successfully created and executed using HTML.

EXPERIMENT 2

SIMPLE WEB APPLICATION USING SERVLET INTERFACE

1. Aim

To develop a simple web application by implementing the Servlet interface and execute it using Apache Tomcat 10.

2. Description

A Servlet is a Java program that runs on a web server and handles client requests. The Servlet interface provides methods such as init(), service(), destroy(), getServletConfig() and getServletInfo().

The service() method processes client requests and generates dynamic responses.

3. Software Requirements

- JDK 17 or later
- Eclipse IDE for Enterprise Java
- Apache Tomcat 10
- Google Chrome

4. Step-by-Step Procedure

1. Open Eclipse IDE.
2. Select File → New → Dynamic Web Project.
3. Enter the project name as ServletApplication.
4. Select Apache Tomcat 10 as the target runtime.
5. Click Finish.
6. Right-click the Java Resources folder.
7. Select New → Class.
8. Enter the class name as SimpleServlet.
9. Implement the Servlet interface.
10. Override all five interface methods.
11. Write the HTML response inside the service() method.
12. Save the program.
13. Right-click the project and select Run As → Run on Server.
14. Select Apache Tomcat 10.
15. Click Finish.
16. Open Google Chrome and enter the servlet URL.

5. Program

File Name: SimpleServlet.java

java

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.annotation.WebServlet;

@WebServlet("/simple")

public class SimpleServlet implements Servlet {
```

```
ServletConfig config;

public void init(ServletConfig config)
        throws ServletException {

    this.config = config;
}

public void service(ServletRequest request,
        ServletResponse response)
        throws ServletException, IOException {
    response.setContentType("text/html");
    PrintWriter out = response.getWriter();
    out.println("<html><body>");
    out.println("<h1>Welcome to Servlet Application</h1>");
    out.println("<h2>Servlet Interface Example</h2>");
    out.println("</body></html>");
}
public void destroy() {
}
public ServletConfig getServletConfig() {
    return config;
}
public String getServletInfo() {
    return "Simple Servlet Application";
}
```

}

6. Expected Output

Welcome to Servlet Application

Servlet Interface Example

Browser URL:

http://localhost:8080/ServletApplication/simple

7. Screenshots

Screenshot 1: Dynamic Web Project Creation

Capture the ServletApplication project in Eclipse.

[Insert Screenshot Here]

Screenshot 2: Java Source Code

Capture the complete SimpleServlet.java program.

[Insert Screenshot Here]

Screenshot 3: Tomcat Server Running

Capture the Eclipse Servers tab showing Tomcat as Started.

[Insert Screenshot Here]

Screenshot 4: Browser Output

Capture the servlet output in Google Chrome.

[Insert Screenshot Here]

8. Result

Thus, a simple web application was successfully developed by implementing the Servlet interface and executed using Apache Tomcat 10.

EXPERIMENT 3

SIMPLE WEB APPLICATION USING GENERICSERVLET

1. Aim

To develop a simple web application using the GenericServlet class and execute it using Apache Tomcat 10.

2. Description

GenericServlet is an abstract class provided by the Jakarta Servlet API. It is protocol-independent and simplifies servlet development.

It provides a service() method that is overridden to process client requests and generate dynamic HTML responses.

3. Software Requirements

- JDK 17 or later
- Eclipse IDE
- Apache Tomcat 10
- Google Chrome

4. Step-by-Step Procedure

1. Open Eclipse IDE.
2. Create a Dynamic Web Project named GenericApplication.
3. Select Apache Tomcat 10 as the target runtime.
4. Click Finish.
5. Right-click the Java Resources folder.
6. Select New → Class.
7. Enter the class name as GenericDemo.
8. Extend the GenericServlet class.
9. Override the service() method.
10. Set the response content type as text/html.
11. Write the HTML response using PrintWriter.
12. Save the program.
13. Right-click the project.
14. Select Run As → Run on Server.
15. Select Apache Tomcat 10.
16. Click Finish.
17. Open Google Chrome and enter the servlet URL.

5. Program

File Name: GenericDemo.java

java

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.annotation.WebServlet;

@WebServlet("/generic")

public class GenericDemo extends GenericServlet {

    public void service(ServletRequest request,
            ServletResponse response)
            throws ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        out.println("<html>");
        out.println("<body>");

        out.println("<h1>Welcome to Generic Servlet</h1>");

        out.println("<h2>My First Generic Web Application</h2>");

        out.println("</body>");
```

```
        out.println("</html>");
    }
}
```

6. Expected Output

Welcome to Generic Servlet

My First Generic Web Application

Browser URL:

http://localhost:8080/GenericApplication/generic

7. Screenshots

Screenshot 1: GenericApplication Project

[Insert Screenshot Here]

Screenshot 2: GenericDemo.java Source Code

[Insert Screenshot Here]

Screenshot 3: Tomcat 10 Running in Eclipse

[Insert Screenshot Here]

Screenshot 4: GenericServlet Browser Output

[Insert Screenshot Here]

8. Result

Thus, a simple web application was successfully developed using the GenericServlet class and executed using Apache Tomcat 10.

EXPERIMENT 4

SIMPLE WEB APPLICATION USING HTTPSERVLET

1. Aim

To develop a simple web application using the HttpServlet class and execute it using Apache Tomcat 10.

2. Description

HttpServlet is an abstract class used to develop HTTP-based web applications. It provides methods such as doGet() and doPost() to handle HTTP requests.

The doGet() method is used to handle GET requests, while the doPost() method is used to handle POST requests.

3. Software Requirements

- JDK 17 or later
- Eclipse IDE
- Apache Tomcat 10
- Google Chrome

4. Step-by-Step Procedure

1. Open Eclipse IDE.
2. Create a Dynamic Web Project named HttpApplication.
3. Select Apache Tomcat 10 as the target runtime.
4. Click Finish.
5. Right-click the Java Resources folder.
6. Select New → Class.
7. Enter the class name as HttpDemo.
8. Extend the HttpServlet class.
9. Override the doGet() method.
10. Set the response content type.
11. Write the HTML response using PrintWriter.
12. Save the program.
13. Right-click the project.
14. Select Run As → Run on Server.
15. Select Apache Tomcat 10.
16. Click Finish.
17. Open the browser and enter the servlet URL.

5. Program

File Name: HttpDemo.java

java

```
import java.io.*;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;

@WebServlet("/http")

public class HttpDemo extends HttpServlet {

    protected void doGet(HttpServletRequest request,
            HttpServletResponse response)
            throws IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        out.println("<html>");
        out.println("<body>");

        out.println("<h1>Welcome to HTTP Servlet</h1>");

        out.println("<h2>HTTP Servlet Web Application</h2>");

        out.println("</body>");
        out.println("</html>");
```

```
    }
}
```

6. Expected Output

Welcome to HTTP Servlet

HTTP Servlet Web Application

Browser URL:

http://localhost:8080/HttpApplication/http

7. Screenshots

Screenshot 1: HttpApplication Project in Eclipse

[Insert Screenshot Here]

Screenshot 2: HttpDemo.java Source Code

[Insert Screenshot Here]

Screenshot 3: Tomcat Server Started

[Insert Screenshot Here]

Screenshot 4: HTTP Servlet Browser Output

[Insert Screenshot Here]

8. Result

Thus, a simple web application was successfully developed using the HttpServlet class and executed using Apache Tomcat 10.

EXPERIMENT 5

RUNNING SERVLET USING APACHE TOMCAT SERVER

1. Aim

To configure Apache Tomcat 10 and execute a Java Servlet application using the Tomcat server.

2. Description

Apache Tomcat is an open-source web server and servlet container. It provides an environment to execute Java Servlet and JSP applications.

Tomcat receives client requests, processes them through the servlet and sends the generated response back to the browser.

3. Software Requirements

- JDK 17 or later
- Eclipse IDE for Enterprise Java
- Apache Tomcat 10
- Google Chrome

4. Step-by-Step Procedure

Step 1: Install Java JDK

1. Download and install JDK.
2. Open Command Prompt.
3. Type the following command:

bash

java -version

4. Verify the installed Java version.

Step 2: Download Apache Tomcat 10

1. Open the official Apache Tomcat website.
2. Download the Tomcat 10 ZIP file.
3. Extract the downloaded ZIP file.
4. Keep the extracted folder in an accessible location.

Step 3: Open Eclipse IDE

1. Start Eclipse IDE.
2. Select a workspace.
3. Open the Enterprise Java development environment.

Step 4: Configure Tomcat Server

1. Open the Servers tab.
2. Click Create a New Server.
3. Select Apache.

4. Select Tomcat v10.0 Server.
5. Click Next.
6. Browse and select the Tomcat installation directory.
7. Click Finish.

Step 5: Create Dynamic Web Project

1. Select File → New → Dynamic Web Project.
2. Enter project name as MyServletProject.
3. Select Apache Tomcat 10 as the target runtime.
4. Click Finish.

Step 6: Create Servlet Class

1. Right-click the Java Resources folder.
2. Select New → Class.
3. Enter the class name as WelcomeServlet.
4. Extend HttpServlet.
5. Add the @WebServlet annotation.

Step 7: Write Java Program

File Name: WelcomeServlet.java

java

```
import java.io.*;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;

@WebServlet("/welcome")

public class WelcomeServlet extends HttpServlet {

   protected void doGet(HttpServletRequest request,
```

```
        HttpServletResponse response)
        throws IOException {

    response.setContentType("text/html");

    PrintWriter out = response.getWriter();

    out.println("<html><body>");

    out.println("<h1>Welcome to Apache Tomcat 10</h1>");

    out.println("<h2>Servlet Executed Successfully</h2>");

    out.println("</body></html>");
  }
}
```

Step 8: Run the Servlet

1. Save the Java program.
2. Right-click the project.
3. Select Run As → Run on Server.
4. Select Apache Tomcat 10.
5. Click Finish.
6. Wait until the server starts successfully.

Step 9: Open the Browser

Enter the following URL:

http://localhost:8080/MyServletProject/welcome

Step 10: Verify the Output

Check whether the browser displays the welcome message.

5. Expected Output

Welcome to Apache Tomcat 10

Servlet Executed Successfully

6. Important Tomcat Commands

To start Tomcat on Windows:

bash

startup.bat

To stop Tomcat:

bash

shutdown.bat

These commands should be executed from the Tomcat bin folder.

7. Screenshots

Screenshot 1: Java Version Verification

Capture the Command Prompt showing the Java version.

[Insert Screenshot Here]

Screenshot 2: Apache Tomcat Installation Folder

Capture the extracted Tomcat 10 folder.

[Insert Screenshot Here]

Screenshot 3: Tomcat Configuration in Eclipse

Capture the server configuration window.

[Insert Screenshot Here]

Screenshot 4: Dynamic Web Project

Capture MyServletProject in Eclipse.

[Insert Screenshot Here]

Screenshot 5: WelcomeServlet.java Source Code

Capture the complete Java program.

[Insert Screenshot Here]

Screenshot 6: Tomcat Server Started

Capture the Eclipse Servers tab showing Tomcat as Started.

[Insert Screenshot Here]

Screenshot 7: Browser Output

Capture the successful servlet output in Google Chrome.

[Insert Screenshot Here]

8. Result

Thus, Apache Tomcat 10 was successfully configured and the Java Servlet application was deployed and executed.